

# HelixAgent

## HelixAgent

하나의 모델을 선택하지 마세요 — 모델들이 토론하게 하고, 그들이 합의한 답변을 제공하세요.

### 요약

HelixAgent는 Go 기반의 프로덕션 준비 완료된 AI 기반 앙상블 LLM 서비스입니다. 여러 언어 모델의 응답을 지능적으로 결합하여 — 다중 라운드 AI 토론 시스템과 동적 검증 기반 제공자 선택을 포함 — 가장 정확하고 신뢰할 수 있는 결과를 생성합니다.

### 간략 설명

HelixAgent는 Go 기반의 앙상블 LLM 서비스로, 여러 제공자를 하나의 정확한 답변으로 통합합니다. 다중 라운드 AI 토론을 진행하고, LLMsVerifier를 통해 제공자를 동적으로 평가하며, 신뢰도 가중치 라우팅 전략을 사용합니다. 또한 캐싱, 모니터링, 보안 가드레일, OpenAI 스타일 API 등 프로덕션 기능을 제공합니다.

### 상세 설명

HelixAgent는 프로덕션 준비 완료된 AI 기반 앙상블 LLM 서비스(MIT 라이선스)로, 단일 모델의 답변을 가설로 간주하며 최종 결론으로 여기지 않습니다. 한 제공자가 틀리거나 편향되거나 일시적으로 이용 불가능할 위험을 감수하기보다는, 여러 언어 모델의 응답을 결합하여 가장 정확하고 신뢰할 수 있는 결과를 도출합니다. 질문이 충분히 복잡할 경우, 모델들을 구조화된 다중 라운드 토론에 참여시켜 최적의 답변을 도출합니다. 지원하는 제공자는 다양하며, README 문서에 `internal/llm/providers/` 하위에 Claude, DeepSeek, Gemini, Mistral, Qwen, xAI/Grok 등 여러 LLM 제공자가 명시되어 있습니다.

핵심은 제공자 선택이 정적인 우선순위 목록에 의존하지 않는다는 점입니다. 통합된 LLMsVerifier의 실시간 검증 점수가 라우팅을 주도하며, 성능이 저하된 제공자는 우아하게 대체됩니다. 또한 오류 발생 시 카테고리별 오류 보고가 제공됩니다. AI 토론 오케스트레이터는 모델 간의 불일치를 신호

로 전환합니다. 다양한 토폴로지(메시, 스타, 체인)를 지원하며, 체계적인 단계 프로토콜(제안 → 비판 → 검토 → 종합)을 따르고, 토론 간 학습을 통해 시스템이 모델 조정을 개선해 나갑니다. 라우팅 전략은 신뢰도 가중치 선택, 다수결 합의, 의미론적 의도 감지 등 다양하며, 실시간 스트리밍 응답을 통해 전체 앙상블이 완료되기 전 토큰 단위로 답변이 전달됩니다.

이 서비스는 데모용이 아닌 실제 프로덕션 환경에서 건널 수 있도록 설계되었습니다. PostgreSQL과 Redis는 고가용성 데이터 레이어를 형성하며, Prometheus/Grafana/OpenTelemetry는 메트릭, 대시보드, 추적 기능을 제공합니다. 또한 JWT 인증, 요청 제한, 가드레일 엔진, PII 탐지 기능이 앙상블을 실제 배포에 필요한 제어 체계로 감쌉니다. 약 20개의 모듈(EventBus, Observability, Auth, Storage, VectorDB, Embeddings, RAG, Memory, MCP 등)로 구성되어 있으며, 각 모듈은 독립적인 역할을 담당합니다. 또한 LLM 최적화 프레임워크(의미론적 캐싱, 구조화된 출력, 향상된 스트리밍)를 제공하며, SGLang, LlamaIndex, LangChain, Guidance, LMQL 등과의 통합을 지원합니다. 완료 및 앙상블 엔드포인트가 OpenAI와 호환되므로, 기존 클라이언트는 HelixAgent로 전환하여 앙상블 추론 기능을 추가로 활용할 수 있습니다.

콘텐츠

## 왜 만들었는가

단일 LLM는 오류를 범하거나, 편향될 수 있으며, 때로는 이용 불가능할 수도 있습니다. HelixAgent는 애플리케이션이 여러 모델을 동시에 참조하고, 그 답변을 측정된 신뢰도에 따라 가중치를 부여하며, 우아하게 대체 모델로 전환할 수 있도록 설계되었습니다. 이를 통해 취약한 단일 제공자에 대한 의존성을 탄력적이고 자가 평가하는 앙상블로 변모시킵니다.

## 혁신적인 이유

이 시스템은 멀티 모델 합의를 실용화합니다. “여러 모델에 질문하고 그 결과를 조율한다”는 개념을 임시 스크립트에서 벗어나 프로덕션 서비스로 전환한 것입니다. 한 제공자에 의존하며 그저 희망하는 대신, 팀은 실시간 검증 점수에 기반한 라우팅, 한 번의 시도만으로는 부족한 질문에 대한 구조화된 토론 프로토콜, 그리고 프로덕션 수준의 탄력성(고가용성 데이터 레이어, 완전한 관측 가능성, 안전장치)을 OpenAI 호환 API 뒤에서 제공합니다. 진정한 변화는 기존 클라이언트가 코드를 수정하지 않고 엔드포인트만 변경함으로써, 취약한 단일 제공자 의존성을 탄력적이고 자가 평가하는 앙상블로 전환할 수 있다는 점입니다.

## 혁신적인 요소

- 모델 간 불일치를 자원으로 활용하는 구조화된 다회전 AI 토론: 선택 가능한 메시/스타/체인 토폴로지, 체계적인 제안→비판→검토→종합 프로토콜, 그리고 시간이 지남에 따라 축적되는 토론 간 학습.

- 정적 선호 목록이 아닌 실시간 LLMsVerifier 점수를 기반으로 한 동적 제공자 선택. 앙상블은 현재 실제로 성능이 좋은 제공자에게 라우팅하며, 한 제공자의 성능이 저하되면 우아하게 대체합니다.
- 자체적으로 기능하는 네이티브 Go LLM 최적화 프레임워크(의미 캐시, 구조화된 출력, 향상된 스트리밍)로, 필요에 따라 외부 최적화 도구(SGLang, LlamaIndex, LangChain, Guidance, LMQL)를 선택적으로 통합할 수 있습니다.
- 약 20개의 모듈로 분리된 모듈형 아키텍처로, 관심사를 명확히 분리하고 분산 메모리나 지식 그래프 스트리밍과 같은 빅데이터 기능을 도입할 수 있는 문을 엽니다.

## 가장 큰 기술적 도전 과제와 해결 방법

- 다양한 품질의 제공자 중 선택하기. 제공자들은 품질이 다르고 시간이 지남에 따라 변화하기 때문에, 고정된 순위는 곧 쓸모없어집니다. 우리는 선택 과정을 지속적으로 측정함으로써 이 문제를 해결했습니다. LLMsVerifier 점수는 신뢰도 가중치 및 다수결 라우팅에 반영되며, 성능이 저하된 제공자는 신뢰하지 않고 우회합니다.
- 진정으로 어려운 질문에 신뢰할 수 있는 답변 얻기. 단일 모델은 한 번의 시도만으로는 자체 오류를 감지할 메커니즘이 없습니다. 토론 오케스트레이터는 이를 해결합니다. 다중 토폴로지와 단계별 토론(제안→비판→검토→종합)을 통해 모델들이 서로의 답변을 검증하고 개선하도록 유도한 후 최종 답변을 종합합니다.
- 노트북이 아닌 프로덕션 환경에서 앙상블 운영하기. 여러 제공자에 분산 요청하면 장애 발생 가능성이 기하급수적으로 증가합니다. 우리는 PostgreSQL+Redis 고가용성 데이터 레이어, Prometheus/Grafana/OpenTelemetry 관측 가능성 도구로 제공자나 라우팅의 이상 징후를 감지하며, JWT 인증, 요청 제한, 안전장치 엔진, PII 탐지 등으로 보안 경계를 구축했습니다.

콘텐츠

## 기술 스택

- **Go** — 단일 요청을 여러 제공업체로 동시에 분산 처리하는 것이 고루틴(goroutine)의 핵심 기능이며, 단일 바이너리 배포로 약 20개 모듈로 구성된 서비스를 간편하게 배포할 수 있다는 점에서 선택되었습니다. 이 기술은 전체 서비스와 모든 내부 모듈의 기반을 이루고 있습니다.
- **Gin (Web API)** — 빠른 응답 속도와 낮은 오버헤드를 제공하는 HTTP 인터페이스로 선택되었습니다. 기존 클라이언트가 변경 없이 앙상블을 활용할 수 있도록 OpenAI와 호환되는 /v1 완료(completion), 채팅, 스트리밍, 앙상블 엔드포인트를 제공합니다.
- **PostgreSQL** — 세션, 분석 데이터, 토론 기록을 영구 저장하는 내구성 있는 스토어로 선택되었습니다. 합의 결정과 토론 이력을 감사할 수 있도록 하며, 고가용성(HA) 데이터 레이어를 구축하는 핵심 요소입니다.
- **Redis** — 낮은 지연 시간의 캐싱과 작업 큐잉을 위해 선택되었습니다. 응답 캐싱과 의미론적 캐시 레이어를 구동하여 반복되거나 유사한 프롬프트의 중복 추론을 건너뛸 수 있게 합니다.

- **LLMsVerifier (통합형)** — 제공업체의 신뢰성을 가정에서 측정 가능한 지표로 전환하기 위해 선택되었습니다. 이 점수를 기반으로 라우팅 우선순위를 정하고, 특정 제공업체의 성능 저하 시 대체 경로를 제공합니다.
- **Prometheus + Grafana + OpenTelemetry** — 여러 제공업체에 걸친 양상불을 관찰 가능하게 유지하기 위해 선택되었습니다. helixagent\_\* 메트릭, 대시보드, 팬아웃(fan-out) 전반의 엔드투엔드 요청 추적을 제공합니다.
- **Model Context Protocol (MCP 어댑터)** — 개방형 프로토콜을 통한 확장성을 위해 선택되었습니다. README에는 외부 도구 및 컨텍스트를 연결하기 위한 다양한 MCP 어댑터가 소개되어 있습니다.
- **Neo4j / ClickHouse / Kafka (빅데이터)** — 단일 노드의 한계를 극복하기 위해 선택되었습니다. Neo4j와 ClickHouse는 분산 메모리와 지식 그래프 기능을 지원하며, Kafka는 이러한 그래프와 이벤트 데이터를 대규모로 스트리밍합니다.
- **최적화 통합 (SGLang, LlamaIndex, LangChain, Guidance, LMQL)** — 접두사 캐싱, 검색, 작업 분해, 제약 생성 등 선택적인 최적화 서비스를 추가할 수 있도록 선택되었습니다. 무거운 최적화 기능은 필요에 따라 활성화할 수 있으며, 필수 사항은 아닙니다.

## 현황 및 투명성 안내

- **현황:** 베타. 이 서비스는 프로덕션 준비 상태라고 설명되지만, README에 기재된 성능 및 커버리지 수치(예: “초당 1000+ 요청 처리”, “캐시 응답 시간 <500ms”, 제공업체 및 검증 스크립트 수 등)는 프로젝트 자체에서 보고한 내용으로, 독립적인 검증은 이루어지지 않았습니다. 이에 따라 본 문서에서는 의도적으로 정성적인 표현을 사용하고 있습니다.
- 제공업체 수는 README 내에서도 상이하게 기재되어 있으며, 본 문서에서는 “다수의 제공업체”라는 정성적 표현을 사용합니다.

우선 순위 계층: Helix-기본.